Assignment 2 for Quality Assurance

Xiyu Du, Zhenqi Liu, Hao Li, Xingrong Zong

December 2025

Part 1 video

https://drive.google.com/file/d/1s8GkuMTnkAYBWyAADSjlplMMMs7y8aJS/view?usp=sharing

Q2.1

This is not a good test suite, which fails to exercise the input domain effectively. It lacks coverage for edge cases, boundary values, and varying input arrangements that are critical for verifying sorting logic.

1.Equivalence Partitioning (EP)

EP is a technique to identify sets of test input that trigger the same SUT behaviour. A good test suite should select at least one representative from each partition.
Missing Partitions:

1. **Null Partition:** The suite lacks a test case to verify if the algorithms throw a proper exception or handle `null` gracefully.
2. **Negative Numbers:** The test does not verify negative integers (e.g., -5,-3,-10)
3. **Duplicate Elements:** The test only verify unique numbers. It should handle arrays with duplicate values (e.g., 1,5,1,5,9)
4. **Large Inputs:** The test should test stack overflow, (e.g., 10,5,10^{20},9)

2.Boundary Value Analysis (BVA)

BVA is a black-box testing technique where we choose pairs of input boundaries that trigger different behaviour.
Missing Boundaries:

1. **Size Boundary (0 vs 1):** The suite lacks tests for empty array (Size=0) input = {} and single element (Size=1): input = {1}
2. **Size Boundary (1 vs multiple):** The suite lacks tests for single element (Size=1) input = {1} and multiple elements (Size¿1): input = {1,3,2}

3.Structural Input Variations (Best/Worst Cases)

In sorting, the arrangement of the data is just as important as the values. Black-box testing for sorting should consider how the specification handles pre-sortedness.

1. **Already Sorted:** Input {1,2,3,4,5}.
2. **Reverse Sorted:** Input {5,4,3,2,1}.

Q2.2

(i)

The test coverage (line coverage) and mutation coverage are shown in Figure 1.

(ii)

1. Relational Operator Replacement

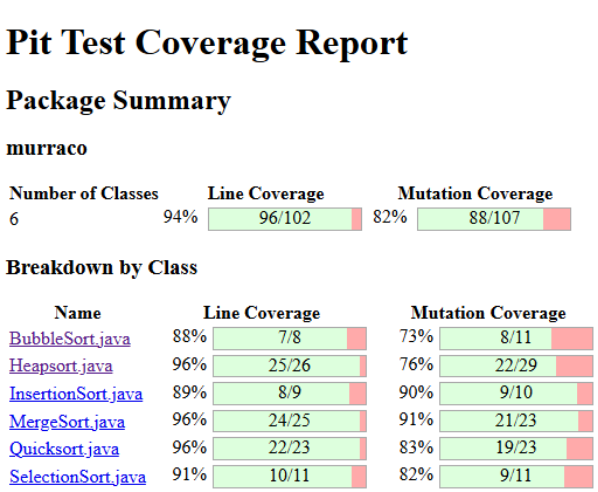
Location: `SelectionSort.java`, line 7.
Mutation: `i < arr.length - 1` to `i <= arr.length - 1`.
Reason of Survive: In Selection Sort, the outer loop iterates to pick the position to fill. The original algorithm stops at `length - 1` because if the first $n-1$ elements are sorted, the last element is automatically in the correct place. The mutant changed the loop to go through the last element, which has already been sorted. Therefore, the result will not change.

2. Arithmetic Operator Replacement

Location: `BubbleSort.java`, line 7.
Mutation: `i < arr.length - 1` to `i < arr.length + 1`
Reason of Survive: Bubble Sort works by swapping adjacent elements that are out of order. After mutating, the loop runs N+2 times. However, once the array is fully sorted, the rest of the loop will only scan the array and make zero swaps.

3. Increments Mutator

Location: `MergeSort.java`, line 31.
Mutation: `arr[i] = left[j++];` to `arr[i] = left[j--];`
Reason of Survive: This block only executes if the `right` array has been completely exhausted before the `left` one. However, in the test suite, the **right** never exhausted first, and the mutation has never been executed. Figure 2. shows that after adding test case, the mutant killed.



Pit Test Coverage Report

Package Summary

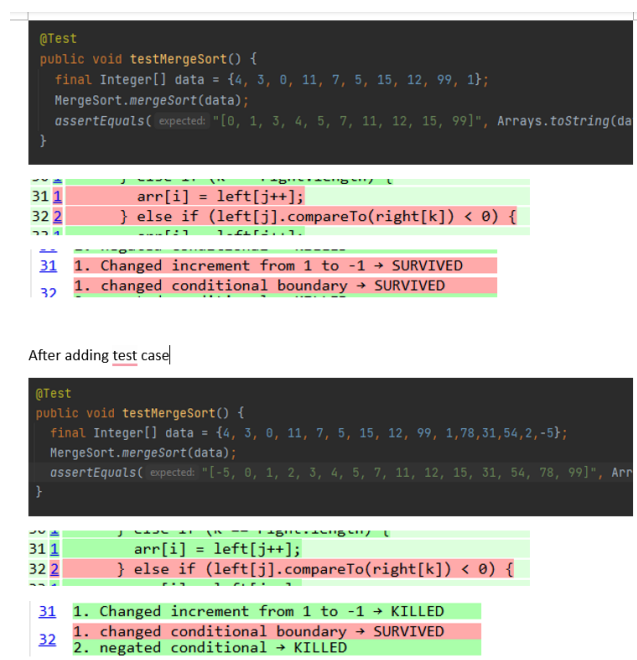
murraco

Number of Classes	Line Coverage		Mutation Coverage	
6	94%	96/102	82%	88/107

Breakdown by Class

Name	Line Coverage		Mutation Coverage	
BubbleSort.java	88%	7/8	73%	8/11
Heapsort.java	96%	25/26	76%	22/29
InsertionSort.java	89%	8/9	90%	9/10
MergeSort.java	96%	24/25	91%	21/23
Quicksort.java	96%	22/23	83%	19/23
SelectionSort.java	91%	10/11	82%	9/11

(a) Figure 1



(b) Figure 2

Q2.3

We designed 10 new test cases applying the black-box techniques in Q2.1 and targeting the surviving mutants in Q2.2.

Test Case 1: testBubbleSortEmpty

- **Technique:** Boundary Value Analysis
- **Justification:** Identified in Q2.1 as a missing boundary. The original test suite lacked a test for input = {} (size = 0). This test case ensures the algorithm handles empty input without errors.

Test Case 2: testInsertionSortSingle

- **Technique:** Boundary Value Analysis
- **Justification:** Identified in Q2.1 as a missing boundary. This test case ensures the loop conditions handle single-element arrays correctly.

Test Case 3: testSelectionSortReverseSorted

- **Technique:** Structural Input Variations: Worst Case
- **Justification:** Identified in Q2.1. Reverse sorted input {5,4,3,2,1} forces the maximum swaps in algorithms in order to verify the worst-case behavior.

Test Case 4: testBubbleAlreadySorted

- **Technique:** Structural Input Variations: Best Case
- **Justification:** Identified in Q2.1. Input{1, 2, 3, 4, 5} tests the "Best case" scenario. For optimized Bubble Sort, this test case should result in $O(n)$ performance.

Test Case 5: testQuickSortNegative

- **Technique:** Equivalence Partitioning
- **Justification:** Identified in Q2.1. The original test suite only used positive integers. This test case verifies whether the algorithm logic holds for negative values like {-5, -3, -10}.

Test Case 6: testAllIdenticalValues

- **Technique:** Equivalence Partitioning
- **Justification:**Identified in Q2.1. An array of identical values {5, 5, 5} tests the "equal" boundary condition for every single comparison. This ensures stability and that `Quicksort` partitions do not degrade into infinite loops or invalid states when all elements are equal to the pivot.

Test Case 7: testDuplicateValues

- **Technique:** Relational Operator Replacement
- **Justification:** Identified in Q2.2, multiple changed conditional boundary mutants survived in `InsertionSort`. These conditionals control how the algorithm handles equal elements. Providing input with duplicates specifically targets {3, 1, 2, 3, 1} these conditions to ensure the algorithm handles equality correctly.

Test Case 8: testMergeSortRightSideExhaustion

- **Technique:** Increments Mutator
- **Justification:** Identified in Q2.2, A mutant in `MergeSort` (j++ changed to j–) survived because the test case never executed the branch where the right sub-array is exhausted first. The input {4, 3, 0, 11, 7, 5, 15, 12, 99, 1, 78, 31, 54, 2, -5, 9} is to ensure the right side values are smaller, and exhausted first, which forces execution of the uncovered line.

Test Case 9: testLargeArray

- **Technique:** Arithmetic Operator Replacement
- **Justification:** Identified in Q2.2, `BubbleSort` has a survivor regarding `i < arr.length + 1`. While this specific mutant might still survive, testing a large array helps ensure that performance-degrading mutants or stack-overflow causing mutants (in recursion) are identified. It validates the algorithm's robustness beyond trivial inputs.

Test Case 10: testHeapsortChildCalculation

- **Technique:** Arithmetic Operator Replacement
- **Justification:** Identified in Q2.2, in `HeapSort`, a mutant replacing + with - (line 18) in the child index calculation survived. This implies the original small dataset didn't trigger a fault even when the index calculation was slightly off. A larger, more complex dataset increases the probability of this calculation error pointing to a value that violates the heap property, thus killing the mutant.

Q2.4

The new tests we created do not reveal any new fault, below we change a line of code to reveal fault.

1. Replace relational operator in `InsertionSort`

Replace `InsertionSort` line 10 from `arr[j - 1].compareTo(temp) > 0)` to `arr[j - 1].compareTo(temp) < 0)`. The test case 7 reveals the fault from expected:[1, 1, 2, 3, 3] to actual:[3, 3, 2, 1, 1]. This is because the original code makes the successful judgment of the smaller value, but the new code is in contrast.

2. Replace arithmetic operator in `MergeSort`

Replace `MergeSort` line 31 from `arr[i] = left[j++];` to `arr[i] = left[j--];`. The test case 8 reveals the fault from Expected:[-5, 0, 1, 2, 3, 4, 5, 7, 9, 11, 12, 15, 31, 54, 78, 99] to Actual:[-5, 0, 1, 2, 3, 4, 5, 7, 9, 11, 12, 15, 31, 54, 78, 31]. The original test cases do not reveal the fault but our new test case does.

3. Replace conditional operator in `InsertSort`

Replace `InsertSort` line 10 from `while (j > 0 || arr[j - 1].compareTo(temp) > 0)` to `while (j > 0 || arr[j - 1].compareTo(temp) > 0)`. The test cases 2 and 7 failed, because of out-of-bounds.

Q2.5

Based on the results, the information of mutants that survived was more useful to create high-value test cases. While black-box techniques help identify obvious missing standard cases, the surviving mutants show the logical gaps which have been missed.
For example, in `MergeSort`, the mutant Changed increment from 1 to -1 survived at line 31. This told us explicitly that the test suite never triggered the scenario where the right sub-array is exhausted before the left one. A black-box approach might randomly hit this, but the mutant report forced us to craft `testMergeSortRightSideExhaustion` to target it specifically.

After creating ten new test cases, we rerun the PITest. The mutation coverage increases from 82% to 83%. Only test case 8 kills a mutation, this is because some mutation can not be killed. For example, as described in Q2.2 relational operator replacement, the sorted list remain same no matter how many times it sort.

Team Contribution

Team Member	**Description of Contributions**
Zhenqi Liu	Video(Section 1, Section 2, Section 3), Part 2 (Q2.1 - Q2.5)
Xiyu Du	Video(Section 1, Section 2, Section 3), Part 2 (Q2.1 - Q2.5)
Hao Li	Video(Section 1, Section 2, Section 3), Part 2 (Q2.1 - Q2.5)
Xingrong Zong	Video(Section 1, Section 2, Section 3), Part 2 (Q2.1 - Q2.5)